

ONE-SHOT PROFESSIONAL HANDBOOK

Modern Web Hosting & Deployment

From DNS to production without the confusion

Cloudflare first | Vercel | Netlify | Hostinger | GitHub Pages

Fundamentals → Hosting choices → CI/CD → Security → Costs → Production

DhruvilExplains

2026 Edition

How the Web Reaches Your Project

Visitor → DNS → CDN / Proxy → Hosting → App / Files / Database

The six parts

- **Domain**: the readable address, such as `example.com`.
- **Registrar**: where the domain is registered and renewed.
- **DNS**: the routing map that connects names to services.
- **Hosting**: the machine or platform serving the project.

- **Origin**: the real server behind a CDN or proxy.
- **CDN**: nearby edge servers that deliver cached content faster.

One project can use many providers

A domain may be registered at one company, use Cloudflare DNS, serve the frontend from Vercel, run an API elsewhere, store files in R2, and use Google or Zoho for email. These parts do not have to live together.

DNS records you must recognize

Record	Mechanical meaning	Common use
A	Name points to IPv4	Root domain to a server IP
AAAA	Name points to IPv6	IPv6-enabled origin
CNAME	Alias points to another hostname	<code>www</code> , app, or hosted platform
MX	Mail exchanger	Incoming company email
TXT	Text proof or policy	Verification, SPF, DKIM, DMARC
NS	Authoritative DNS control	Delegates DNS to a provider
CAA	Certificate-authority permission	Restricts who may issue TLS certificates

Safe migration rule. Change only the website records you understand. Preserve MX and email-related TXT/CNAME records. Moving a website must not silently break company email.

Root, subdomain and propagation

- `@` means the zone apex or root: `example.com`.
- `www`, `app`, and `api` are separate hostnames.
- DNS changes are cached according to **TTL**; different networks may update at different times.
- A DNS record routes traffic. It does not upload or build your code.

Cloudflare: The First Platform to Understand

Cloudflare can act as DNS provider, reverse proxy, CDN, security layer, serverless runtime, and object-storage provider. You can use only the parts you need.

Orange cloud: proxied

Visitor → Cloudflare → origin

Cloudflare can terminate HTTPS, filter attacks, cache eligible responses, hide the origin IP, and add analytics. Common for web traffic.

Grey cloud: DNS only

Visitor → origin directly

Cloudflare returns the DNS answer but does not proxy the connection. Mail infrastructure and unsupported protocols are commonly DNS-only.

TLS modes

Mode	Connection	Verdict
Off	No HTTPS protection	Never for production
Flexible	HTTPS to Cloudflare, HTTP to origin	Avoid; can create redirect loops
Full	HTTPS both sides; origin identity not strictly checked	Temporary compatibility
Full (strict)	HTTPS both sides with valid origin certificate	OK Recommended

Enable HSTS only after HTTPS works on every required subdomain. Browsers remember HSTS, so a bad configuration is harder to undo than a normal redirect.

Workers, Pages and R2

Product	Use it for	Do not confuse it with
Workers	APIs, middleware, redirects, edge logic, static assets	A permanently running VPS
Pages	Git-based frontend deployment and previews	A general Linux server
R2	Images, CAD, PDFs, backups, private objects	A relational database
KV	Fast configuration and cached values	Source of truth for transactions
D1	Cloudflare-managed SQL workloads	Universal replacement for every database
Queues	Reliable asynchronous messages	The worker that performs heavy processing

Caching language

HIT served from edge cache **MISS** fetched from origin **TTL** cache lifetime **PURGE** invalidate cached content

Cache fingerprinted CSS, JavaScript, fonts and images aggressively. Do not broadly cache authenticated dashboards, checkout pages, private files, or personalized API responses.

Choose the Correct Hosting Model

Model	Best fit	Operational reality
Static hosting	Portfolio, documentation, landing page, SPA, exported Next.js	Prebuilt files; low attack surface and low cost
Serverless edge /	Forms, webhooks, APIs, auth checks, scheduled jobs	Pay/limit by requests, duration, CPU, bandwidth or credits
Managed app host	SSR, Node apps, APIs, background services	Platform handles deployment; runtime limits still matter
Shared hosting	PHP/WordPress and small conventional sites	Simple panel; resources shared with other tenants
VPS / cloud VM	Long-running backend, custom software, sockets, workers	You patch, secure, monitor and scale the server
Containers	Repeatable services and complex deployments	More control, more operational responsibility

Frontend, full-stack and heavy processing

Static frontend

HTML/CSS/JS or build output. Host on Cloudflare, Netlify, Vercel or GitHub Pages.

Full-stack app

Frontend + server runtime + database + secrets. Use a platform that supports the required runtime.

Heavy compute

Video, CAD, ML, long jobs. Use queues plus dedicated workers, containers or VMs; avoid forcing everything into short serverless calls.

Decision shortcut

Only public files? Static hosting. **Short request logic?** Functions or Workers. **Database + login?** Full-stack platform. **Long CPU/GPU job?** Queue + compute service. **Need OS control?** VPS/container.

Framework output matters

- Vite/React usually builds to `dist/`.
- Next.js static export commonly produces `out/` when configured for export.
- Next.js SSR produces server/runtime output, not merely static HTML.
- A host must receive the exact build command and output directory it expects.

A successful local development server is not a production test. Always run lint, the production build, and a production-like preview before deploying.

The Main Platforms at a Glance

Platform	Strongest fit	Free-entry reality	Watch carefully
Cloudflare	DNS, CDN, security, edge apps, static assets, object storage	Several products have free allowances; quotas and billing models differ	Worker CPU/requests, builds, R2 operations/storage, third-party origin egress
Vercel	Next.js, previews, frontend teams	Hobby is intended for personal, non-commercial use; paid plans support professional use	Function usage, data transfer, image optimization, team and plan rules
Netlify	Frontend workflows, previews, forms, functions	Free/credit-based entry plans have metered allowances	Credits, bandwidth, builds, functions, forms and add-ons
Hostinger	Shared hosting, WordPress, VPS and bundled site services	Usually paid hosting with introductory offers rather than a permanent general free tier	Renewal price, plan term, CPU/RAM/storage, backups, email, migration and refund terms
GitHub Pages	Public static project sites and documentation	GitHub-hosted static publishing within documented usage limits	Static-only architecture, repository/account rules, soft limits, no private backend secrets

What each platform changes mechanically

Cloudflare

Traffic often passes through its proxy before reaching the origin. Its value is not only hosting: DNS, TLS, CDN, WAF, Workers and R2 can form an edge architecture.

Vercel

Git push becomes a preview or production deployment. Framework-aware infrastructure turns routes into static output, functions or edge code depending on the app.

Netlify

Git-driven builds publish immutable deploys. Redirects, forms, functions, preview deploys and rollbacks support frontend teams.

Hostinger

Traditional plans may provide a control panel, files, databases, WordPress and mail. VPS plans provide a server you must operate more directly.

GitHub Pages

A branch or Actions workflow publishes static files. It is excellent for docs and simple sites, but it is not an application server.

Pricing language that stays accurate: a free plan is a limited allowance, not a promise of unlimited or permanent zero cost. Plans, quotas, regional taxes and acceptable-use rules change. Verify the provider's current pricing and terms before a commercial launch.

GitHub Deployment and CI/CD

GitHub stores source code; a hosting provider runs a build and publishes the result. GitHub Actions can also perform the build and deployment.

Code change → commit → push → CI checks → build artifact → deploy → health check → rollback if needed

Healthy branch flow

- Feature branch: isolated work.
- Pull request: review and automated checks.
- Preview: test the real build before merging.
- `main`: protected production source.
- Release/tag: traceable production milestone.

Minimum pipeline

1. Install locked dependencies.
2. Run formatting/lint checks.
3. Run tests.
4. Create the production build.
5. Deploy only the generated artifact.
6. Verify the public URL.

Typical build settings

Project	Build command	Publish/output
Plain static site	none or project script	repository folder
Vite frontend	<code>npm run build</code>	<code>dist</code>
Next.js static export	<code>npm run build</code>	<code>out</code>
Next.js SSR	platform adapter/build	runtime-specific output
Cloudflare static Worker	build, then <code>wrangler deploy</code>	asset directory in Wrangler config

GitHub Actions: the mental model

A workflow file in `.github/workflows/` defines triggers, jobs and steps. Use repository secrets for deployment tokens. Pin permissions to the minimum required. Prefer short-lived identity federation when supported rather than permanent cloud keys.

Do not place secrets in source code, client-side JavaScript, build logs, screenshots, or variables prefixed for browser exposure. A secret used by frontend code is no longer secret.

Costs: Bandwidth, Egress, Credits and Limits

The cost equation

Monthly cost $\approx$ plan + builds + compute + database + storage + operations + data transfer + domains/email + add-ons + taxes

Ingress and egress

Ingress is data entering a platform. **Egress** is data leaving it for users or another provider. Egress is often the more important billable direction.

Example: a 20 MB download served 10,000 times is roughly 200 GB of outbound data before

protocol and caching effects.

Credits are not universal units

Some providers combine different usage into credits; others meter bandwidth, requests, execution duration, CPU time, storage or operations separately. Read the unit, included allowance, overage behavior and reset period.

Questions to answer before publishing

- Is commercial use allowed on this plan?
- What happens at the limit: stop, throttle, or charge overage?
- Is a payment method required? Are spending caps or alerts available?
- Are preview builds, image optimization and function invocations metered?
- Is bandwidth measured from the edge, origin, or both?
- Does object storage charge for capacity, operations, retrieval or egress?
- Is the attractive price introductory? What is the renewal price and term?
- Are email, backups, SSL, migration and support included or separate?

Free does not mean unrestricted

Phrase	Correct interpretation
Free tier	A quota with eligibility and acceptable-use conditions
No egress fee	The named product may waive a specific transfer charge; other costs can remain
Unlimited bandwidth	Usually subject to fair-use, plan scope or resource policies
Free SSL	Certificate issuance can be free; configuration and origin security still matter
Free domain	Often bundled for a term; renewal and transfer rules still apply

Set budget alerts before traffic arrives. Keep large downloads behind caching or object storage designed for delivery. Compress images, fonts and bundles. Measure real usage instead of assuming a marketing label describes the full bill.

Production Security Without Theatre

Transport and edge

- TLS end-to-end; use strict certificate validation.
- Redirect HTTP to HTTPS once the certificate path works.
- WAF rules for real threats, not random blocking.
- Rate limits on login, forms and expensive APIs.
- DDoS protection and hidden origin where appropriate.

Application

- Validate and normalize input on the server.
- Authorize every private resource; login alone is insufficient.

- Use secure, HttpOnly, SameSite cookies where suitable.
- Verify payment and webhook signatures.
- Patch dependencies and remove unused services.

Secrets and environments

Separate development, preview/staging and production. Give each its own credentials. Rotate leaked secrets immediately; deleting them from the latest commit does not erase Git history.

Headers

Use CSP, HSTS, Referrer-Policy, Permissions-Policy and X-Content-Type-Options intentionally. A copied CSP can break analytics, fonts, APIs, WASM or required scripts.

Private uploads

For customer documents or large files:

Authenticated user → API authorizes → short-lived signed URL → direct object-storage upload → database stores metadata → background job processes file

Keep buckets private by default. Use generated object keys rather than trusting original filenames. Validate size and type, scan when appropriate, set retention rules, and log access without logging confidential contents.

Backups and recovery

A deployment rollback is not a database backup. Define recovery point objective (acceptable data loss) and recovery time objective (acceptable downtime). Test restoration, not only backup creation.

Debugging: Follow the Request Path

Symptom	Likely layer	First checks
DNS error	Registrar/DNS	Nameservers, record name/value, TTL
404	Routing/build output	Publish directory, SPA fallback, route existence
Redirect loop	TLS/redirect rules	Flexible mode, duplicate redirects, canonical host
500	Application/runtime	Function logs, environment variables, dependencies
502/504	Origin/upstream	Process health, timeout, network path
522	Cloudflare to origin	Origin reachability, firewall, IP/port
525/526	TLS to origin	Certificate validity, hostname, strict mode
CORS error	Browser/API policy	Allowed origin, method, headers, credentials
Old content	Cache/deployment	Active deploy, cache headers, purge scope
Build failure	CI/build system	Lockfile, Node version, TypeScript, missing env

Inspection order

1. Confirm the exact URL, environment and time of failure.
2. Resolve DNS and inspect the certificate.
3. Confirm the newest commit reached the provider.
4. Read install, build and deploy logs separately.
5. Verify the output directory and routing rules.
6. Inspect browser Network and Console panels.
7. Check runtime logs, firewall events and upstream health.
8. Compare with the last known-good deployment; roll back when impact is active.

Good incident note: what users observed, when it began, which component failed, evidence found, immediate mitigation, permanent fix, and the test that proves recovery.

Production Release Checklist

- Domain ownership and renewal confirmed
- DNS records exported or documented
- Email records preserved
- HTTPS and canonical redirect tested
- Production build and routes verified
- Environment variables separated
- No secrets in client bundle or repository
- Error pages and SPA fallback tested
- Forms protected and server-verified
- Security headers tested, not blindly copied
- Backups and restore procedure tested
- Logs, alerts and spending controls enabled
- Accessibility and mobile checks completed
- Analytics/privacy choices documented
- Rollback owner and method known
- Current plan terms reviewed

One-Page Architecture and Final Revision

Public web layer

Domain + DNS → TLS + CDN/WAF → static frontend or SSR host

Application layer

Browser → API → authentication/authorization → database

File and job layer

Signed upload → object storage → queue → background processor

Delivery layer

Git branch → checks → build artifact → preview → production → monitoring/rollback

What to choose

Need	Sensible starting point
Documentation or portfolio	GitHub Pages or static platform
React/Vite company site	Cloudflare, Netlify or Vercel static deployment
Next.js with SSR	Vercel or another verified Next.js runtime; Cloudflare via supported adapter
WordPress/site panel	Managed WordPress or shared hosting such as Hostinger
Custom long-running Node service	Managed container/app host or VPS
Private large files	Object storage with signed URLs
Heavy background processing	Queue plus dedicated worker/container
Global DNS, CDN and edge security	Cloudflare in front of a compatible origin

Words to remember

DNS routes

CDN delivers

TLS encrypts

CI builds

CD deploys

Monitoring proves

Final professional rule. Architecture is a set of trade-offs. Choose the smallest platform that satisfies runtime, security, reliability and cost requirements; document the limits; and keep an exit path through standard code, data exports and backups.

Use and accuracy note

Original educational material: free to reuse and adapt for learning or professional revision with attribution to **DhruvilExplains**. Provider names remain their owners' trademarks. Plans, quotas and terms change, so verify official sources before a commercial launch.

Build small. Measure clearly. Scale deliberately.

A compact reference from first DNS record to production recovery.